

Universidad de Jaén
Departamento de Informática

Tema 5

Procesamiento Distribuido de Consultas

Bases de Datos Distribuidas

Grado en Ingeniería Informática

CC BY NC ND

Profesor
Álvaro Labella Romero

Contenidos

- Procesamiento de consultas
- Descomposición de consultas

PROCESAMIENTO DE CONSULTAS

Introducción

- Una operación de acceso por una aplicación se expresará como una consulta que referencia relaciones globales (como si los datos estuvieran en un solo lugar)
- Se aplican **transformaciones de equivalencia** → reglas algebraicas que pueden ser aplicadas a una consulta para reescribirla en una expresión equivalente. Se usarán para simplificar las expresiones de las consultas:
  - Eliminar operaciones innecesarias, reordenar JOINS, combinar proyecciones, etc.
- Se generan **consultas equivalentes** sobre los fragmentos que obtienen los **mismos resultados** que la original
- El SGBDD transforma las consultas simplificadas en **consultas que referencien únicamente a fragmentos ->** se sustituye cada relación global por los fragmentos donde están los datos

PROCESAMIENTO DE CONSULTAS

Objetivos

- El procedimiento se origina → **Procesador de consultas**
  - Transformación (correcta y eficiente) de una consulta de alto nivel (global o sobre fragmentos) en otra equivalente de bajo nivel y decide cómo ejecutarla
  - Transformación a bajo nivel
    - Selección, proyección, JOIN, unión…
  - Implementa la estrategia de ejecución
    - Elegir la mejor consulta más eficiente:
      - Menor tráfico en la red -> consulta a los nodos
      - Menor latencia -> tiempo de respuesta
      - Menor carga en los servidores -> que ejecutan la consulta

PROCESAMIENTO DE CONSULTAS

- **Procesamiento de consultas distribuidas**
  - El tiempo de procesamiento de una consulta distribuida dependerá
    - N° de fragmentos
    - Distribución de los fragmentos en diferentes sitios
    - La velocidad de conexión
    - Capacidad de procesamiento que tenga cada nodo

PROCESAMIENTO DE CONSULTAS

▶ Procesamiento de consultas distribuidas

▷ Medidas para el consumo de recursos:

- Coste total de procesar una consulta:
  Suma de todos los tiempos que se emplean al procesar las operaciones de la consulta en las diversas localidades, más el tiempo necesario para la comunicación entre localidades
  - Comunicación de la consulta a los nodos donde están los datos
  - Ejecución de cada nodo sobre los los fragmentos
  - Unificar datos de diferentes sitios
  - Enviar los resultados al usuario o aplicación
- Tiempo de respuesta de la consulta:

  Tiempo que transcurre en la ejecución de la consulta. Se mide como el tiempo necesario para devolver la primera fila del conjunto de resultados al usuario

  Diferencia clave: el coste total mide **todo el tiempo** empleado en la consulta y el tiempo de respuesta mide el tiempo hasta que el **usuario obtiene el primer dato -> Coste total ≥ Tiempo de respuesta**

PROCESAMIENTO DE CONSULTAS

▶ Etapas

- ▷ Análisis léxico y sintáctico: La consulta escrita en **un lenguaje de alto nivel** (como SQL) es analizada para verificar si es **correcta** en términos de **sintaxis** y **estructura léxica ->** símbolos válidos (SELECT, FROM…), existe tabla, el atributo…
  - Resultado: consulta bien formada.
- ▷ Descomposición de consultas: La consulta se traduce a una **forma algebraica** más simple. Se dividen las operaciones en pasos más básicos (como selecciones, proyecciones y JOINS).
  - Resultado: consulta algebraica simplificada.
- ▷ Localización de datos: Se identifican en qué **fragmentos y servidores** están los datos referenciados en la consulta.
  - Resultado: Consulta fragmentada.
- ▷ Optimización global: Se buscan estrategias para **ejecutar la consulta con el menor coste posible**.
  - Resultado: Consulta optimizada a nivel de fragmentos.
- ▷ Optimización local: Cada nodo que tiene un fragmento de datos realiza una **optimización interna** para ejecutar su parte de la consulta e la mejor forma.
  - Resultado: subconsultas locales optimizadas.

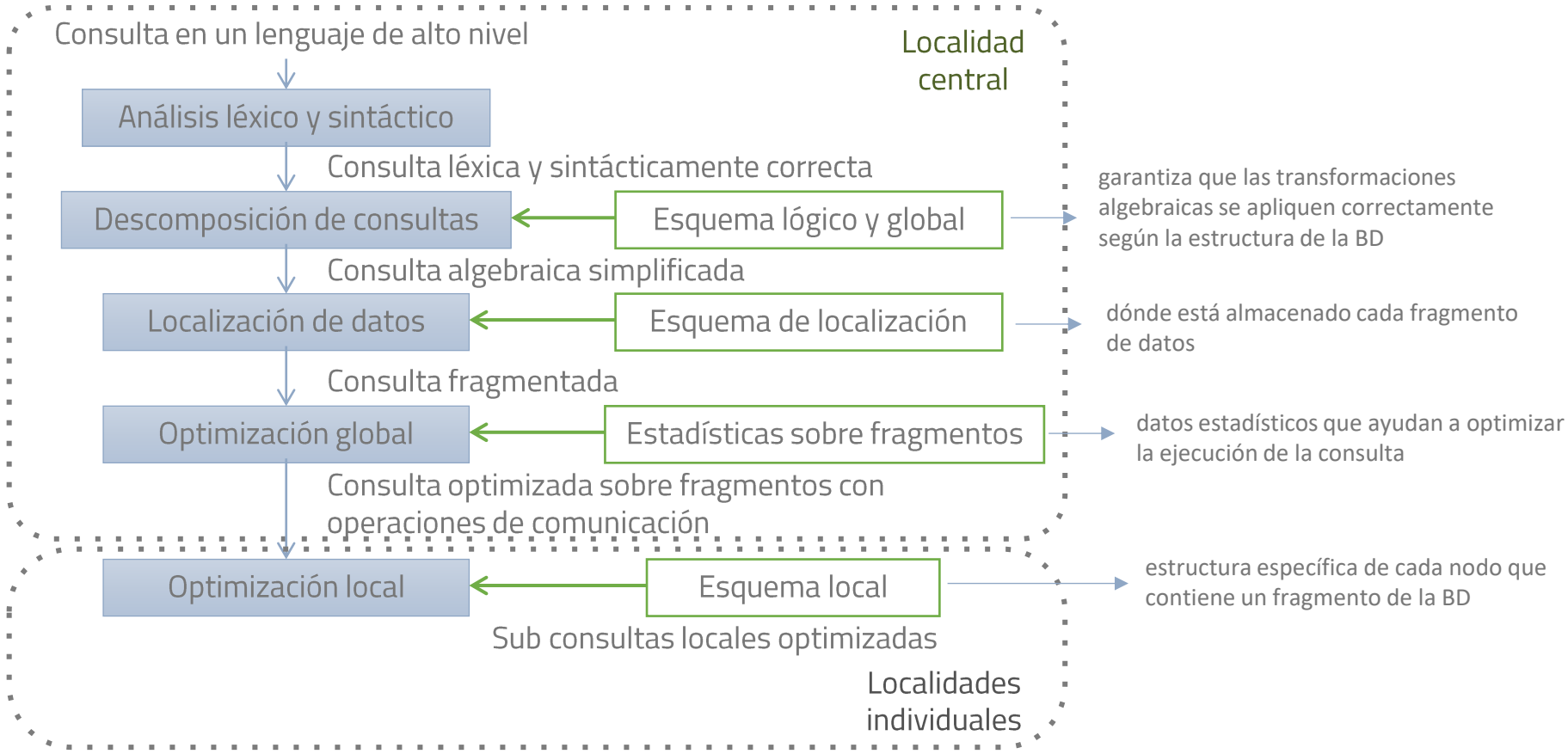
PROCESAMIENTO DE CONSULTAS

▶ Procesamiento de consultas distribuidas

▷ Proceso de mapeo de Consultas Globales a Consultas Locales:

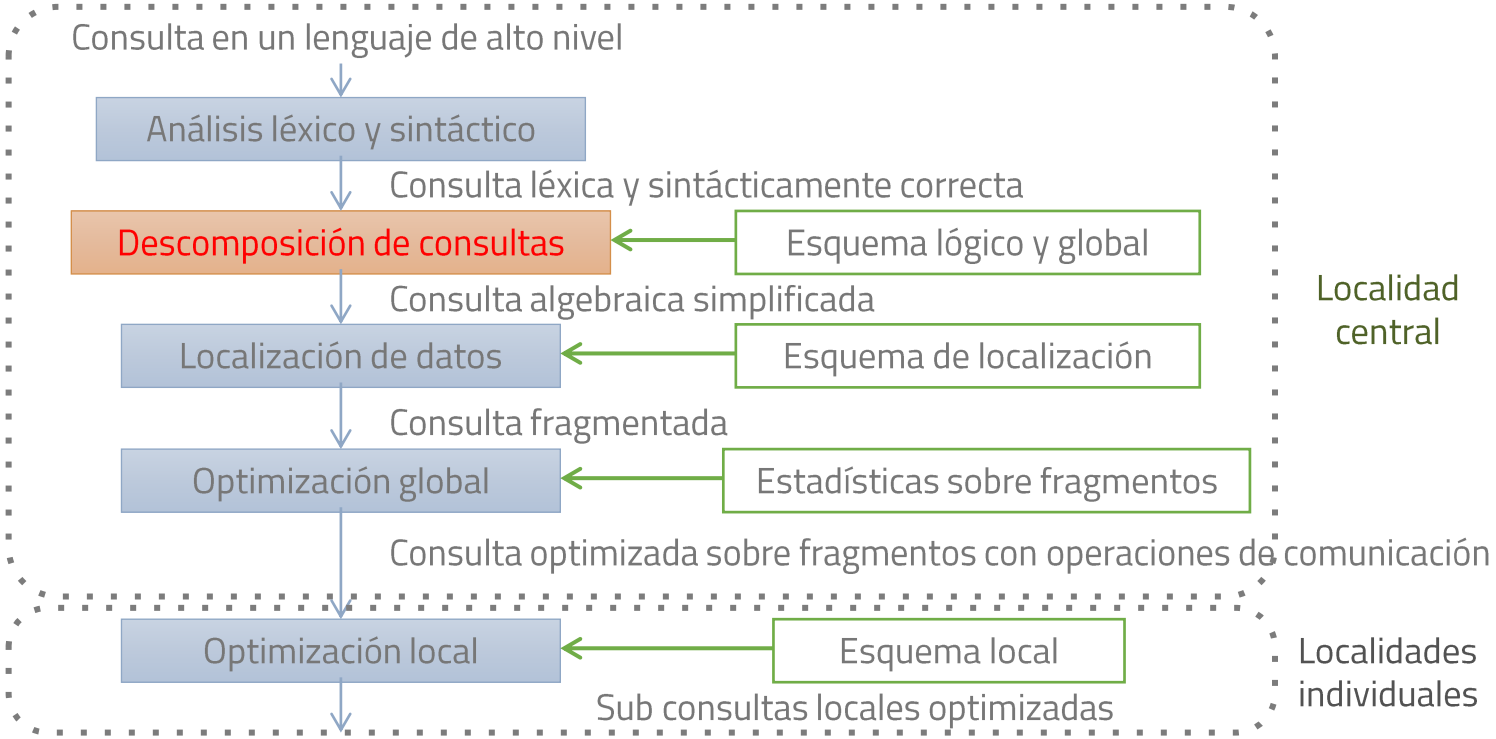
- Las tablas necesarias en una consulta global tienen fragmentos distribuidos en varios sitios. Se utiliza el **diccionario de datos global** para recopilar información sobre la distribución y reconstruye la vista global a partir de los fragmentos.
- Si hay replicación, el **optimizador global** selecciona el sitio en función del coste de comunicación, la carga de trabajo y la velocidad del servidor.
- El **optimizador global** genera un plan de ejecución distribuido para que se produzca la menor transferencia de datos entre los sitios.
- Los **servidores de bases de datos locales** optimizan las consultas locales.
- Los resultados de las consultas locales se fusionan mediante una operación de unión en el caso de los fragmentos horizontales y una operación de JOIN para los fragmentos verticales.

PROCESAMIENTO DE CONSULTAS



PROCESAMIENTO DE CONSULTAS

▶ Etapas





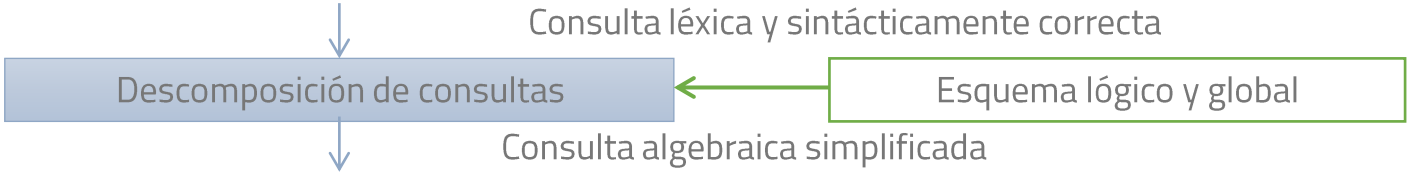
Descomposición de consultas

Bases de Datos Distribuidas

Grado en Ingeniería Informática

CC BY NC ND

Descomposición de consultas



- **Normalización**: Expresar los predicados en forma normal conjuntiva o forma normal disyuntiva
- **Validación**: Detectar y rechazar posibles consultas sintácticamente correctas pero inconsistentes (Ej. edad > 30 AND edad < 20)
- **Simplificación**: Eliminar redundancias
- **Reestructuración**: Reescritura de la consulta en términos del álgebra relacional

Descomposición de consultas

Normalización

- Transformar la consulta a una forma estándar para facilitar el procesamiento posterior
- Dos posibles formas:
  - Forma normal disyuntiva (FND): disyunción de conjunciones

    La consulta se puede procesar como sub **consultas independientes** enlazadas por uniones. Muy útil para fragmentación horizontal.

    $$(p_{11} \wedge \ldots \wedge p_{1n}) \vee \ldots \vee (p_{m1} \wedge \ldots \wedge p_{mn})$$

  - Forma normal conjuntiva (FNC): conjunción de disyunciones

    Es más práctica cuando se necesitan **filtrar datos de fragmentos**

    $$(p_{11} \vee \ldots \vee p_{1n}) \wedge \ldots \wedge (p_{m1} \vee \ldots \vee p_{mn})$$

Descomposición de consultas

- **Forma normal disyuntiva**
  - Nodo Madrid -> clientes Madrid
  - Nodo Sevilla -> clientes Sevilla
  - Consulta: buscar clientes de Madrid mayores de 30 o de Sevilla con ingresos mayores de 40000

$$(ciudad =' Madrid' \wedge edad > 30) \vee (ciudad =' Sevilla' \wedge ingresos > 40000)$$

  - Cada bloque va directo a un nodo

```
ciudad='Madrid' AND edad>30

ciudad='Sevilla' AND ingresos>40000
```

Dos casos separados y dos fragmentos distintos
FND es útil cuando cada disyunción representa caminos independientes de ejecución

Descomposición de consultas

- **Forma normal disyuntiva** -> puede replicar predicados de selección
  - Al expandir una consulta, el mismo predicado puede aparecer repetido en varios términos

$$(edad > 30 \wedge ciudad =' Madrid') \vee (edad > 30 \wedge ciudad =' Sevilla')$$

  - edad > 30 aparece dos veces. El mismo filtro se repite
  - En SQL sería algo así:

```
(edad > 30 AND ciudad='Madrid')
OR
(edad > 30 AND ciudad='Sevilla')
```

Descomposición de consultas

- **Forma normal conjuntiva**
  - Nodo Madrid -> clientes Madrid
  - Nodo Sevilla -> clientes Sevilla
  - Consulta: buscar clientes de Madrid o Sevilla mayores de 30 años (aquí la idea es aplicar un filtro común que aplicar a todos los fragmentos)

$$(ciudad =' Madrid' \vee ciudad =' Sevilla') \wedge edad > 30$$

  - Aparece un filtro común que puedes aplicar a ambos nodos de forma uniforme

```
edad > 30
```

Descomposición de consultas

- **Forma normal conjuntiva** -> puede aparecer replicación si hay muchas disyunciones

$$(p_1 \vee p_2 \vee p_3) \wedge (q_1 \vee q_2)$$

- Al expandirlo para localizar fragmentos o generar subconsultas:

$$(p_1 \wedge q_1) \vee (p_1 \wedge q_2) \vee (p_2 \wedge q_1) \vee \ldots$$

- Se obtienen muchas combinaciones y explota el número de términos. Muchos OR generan muchas combinaciones repetidas

Descomposición de consultas

Reglas de equivalencia de operadores lógicos

1. $p_1 \wedge p_2 \Leftrightarrow p_2 \wedge p_1$ → Propiedad conmutativa conjunción
2. $p_1 \vee p_2 \Leftrightarrow p_2 \vee p_1$ → Propiedad conmutativa disyunción
3. $p_1 \wedge (p_2 \wedge p_3) \Leftrightarrow (p_1 \wedge p_2) \wedge p_3$ → Propiedad asociativa conjunción
4. $p_1 \vee (p_2 \vee p_3) \Leftrightarrow (p_1 \vee p_2) \vee p_3$ → Propiedad asociativa disyunción
5. $p_1 \wedge (p_2 \vee p_3) \Leftrightarrow (p_1 \wedge p_2) \vee (p_1 \wedge p_3)$ → Propiedad distributiva de la conjunción respecto a la disyunción
6. $p_1 \vee (p_2 \wedge p_3) \Leftrightarrow (p_1 \vee p_2) \wedge (p_1 \vee p_3)$ → Propiedad distributiva de la disyunción respecto a la conjunción
7. $\neg (p_1 \wedge p_2) \Leftrightarrow \neg p_1 \vee \neg p_2$ → Ley de pan Morgan para conjunción
8. $\neg (p_1 \vee p_2) \Leftrightarrow \neg p_1 \wedge \neg p_2$ → Ley de pan Morgan para disyunción
9. $\neg (\neg p) \Leftrightarrow p$ → Doble negación

Descomposición de consultas

Validación

- Permite rechazar consultas normalizadas para las que el procesamiento es imposible o innecesario
- Consulta de tipo incorrecto:

  Si las operaciones que se aplican a los atributos no son del tipo correcto
- Consulta semánticamente incorrecta:

  Si cualquiera de sus atributos o nombre de relaciones no están definidos en el esquema
- Ejemplo:

  SELECT E# FROM Empleado WHERE nombre > 200

  - E# no es un atributo declarado en el esquema
  - La operación “>200” es incompatible con el tipo del atributo nombre

DESCOMPOSICIÓN DE CONSULTAS

▶ Simplificación: eliminar redundancias

▷ Las redundancias pueden eliminarse simplificando con las siguientes reglas de idempotencia:

1. $p \wedge p \Leftrightarrow p$ -> (*edad* > *30 AND edad* > *30*) -> *edad* > *30*
2. $p \vee p \Leftrightarrow p$ -> (*ciudad* = *'Madrid' OR ciudad* = *'Madrid'*) -> *ciudad* = *'Madrid'*
3. $p \wedge true \Leftrightarrow p$ -> (*edad* > *30 AND TRUE*) -> *edad* > *30*
4. $p \vee false \Leftrightarrow p$ -> (*edad* > *30 OR FALSE*) -> *edad* > *30*
5. $p \wedge false \Leftrightarrow false$ -> (*edad* > *30 AND FALSE*) -> *consulta vacía*
6. $p \vee true \Leftrightarrow true$ -> (*edad* > *30 OR TRUE*) -> *todos los registros*
7. $p \wedge \neg p \Leftrightarrow false$ -> (*edad* > *30 AND edad* <= *30*) -> *consulta vacía*
8. $p \vee \neg p \Leftrightarrow true$ -> (*edad* > *30 OR edad* <= *30*) -> *todos los registros*
9. $p_1 \wedge (p_1 \vee p_2) \Leftrightarrow p_1$ -> *edad* > *30 AND* (*edad* > *30 OR ciudad* = *'Madrid'*) -> *edad* > *30*
10. $p_1 \vee (p_1 \wedge p_2) \Leftrightarrow p_1$ -> *edad* > *30 OR* (*edad* > *30 AND ciudad* = *Madrid'*) -> *edad* > *30*

DESCOMPOSICIÓN DE CONSULTAS

- Ejemplo

```
SELECT Título
FROM Empleado
WHERE
    (NOT (Título = 'Programador')
     AND (Título = 'Programador' OR Título = 'Ing Mecánica')
     AND NOT (Título = 'Ing Mecánica'))
    OR Nombre = 'Susana';
```

  - Predicados:
    - p_1: Título=“Programador”
    - p_2: Título=“Ing. Mecánica”
    - p_3: Nombre=“Susana”
  - Cualificación de la consulta:

$$(\neg p_1 \wedge (p_1 \vee p_2) \wedge \neg p_2) \vee p_3$$

DESCOMPOSICIÓN DE CONSULTAS

```
SELECT Título
FROM Empleado
WHERE
    (NOT (Título = 'Programador')
     AND (Título = 'Programador' OR Título = 'Ing Mecánica')
     AND NOT (Título = 'Ing Mecánica'))
    OR Nombre = 'Susana';
```

- Ejemplo $(\neg p_1 \wedge (p_1 \vee p_2)) \wedge \neg p_2) \vee p_3$

R9

$p_1 \wedge (p_2 \vee p_3) \Leftrightarrow (p_1 \wedge p_2) \vee (p_1 \wedge p_3)$ → $((\neg p_1 \wedge p_1) \vee (\neg p_1 \wedge p_2)) \wedge \neg p_2 \vee p_3$

R7

$p \wedge \neg p \Leftrightarrow \text{false}$ → $\text{false} \vee (\neg p_1 \wedge p_2)) \wedge \neg p_2 \vee p_3$

R4

$p \vee \text{false} \Leftrightarrow p$ → $(\neg p_1 \wedge p_2) \wedge \neg p_2 \vee p_3$

R7

$p \wedge \neg p \Leftrightarrow \text{false}$ → $(\neg p_1 \wedge p_2) \wedge \neg p_2 \equiv \neg p_1 \wedge (p_2 \wedge \neg p_2)$ → $\neg p_1 \wedge \text{false} \vee p_3$

R5

$p \wedge \text{false} \Leftrightarrow \text{false}$ → $\text{false} \vee p_3$

R4

$p \vee \text{false} \Leftrightarrow p$ → p_3 → `SELECT Título FROM Empleado WHERE Nombre = 'Susana';`

DESCOMPOSICIÓN DE CONSULTAS

Reescritura

- Transformación de la consulta de alto nivel al álgebra relacional para optimizarla
- Interpretación de una expresión de álgebra relacional:
  - Semántica de la consulta -> Qué resultado quiero obtener
  - Secuencia de operaciones

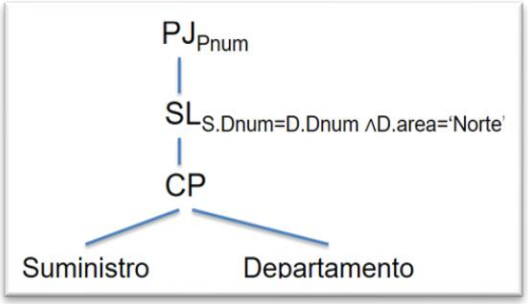


- Así, dos expresiones con la misma semántica se pueden describir con dos secuencias de operaciones diferentes. Mismo resultado pero diferente eficiencia.
- Ejemplo:

$PJ_{Nombre,Dnum}\ SL_{Dnum=7}$ Empleado -> Primero filtramos y luego proyectamos

$SL_{Dnum=7}\ PJ_{Nombre,Dnum}$ Empleado -> Primero proyectamos y luego filtramos

DESCOMPOSICIÓN DE CONSULTAS



Reescritura

Árbol de operadores:

- Es una manera de representar una consulta en álgebra relacional gráficamente y ver el orden en el que se ejecutan las operaciones
- Nodos hoja → relaciones de entrada, disponibles en la cláusula FROM
- Nodo raíz → operación de proyección sobre los atributos correspondientes, especificados en la cláusula SELECT
- Nodos internos → relaciones intermedias producidas por un operador (unario o binario) del álgebra relacional, según la cualificación especificada en la cláusula WHERE
- Orden de ejecución secuencial → desde las hojas hasta la raíz

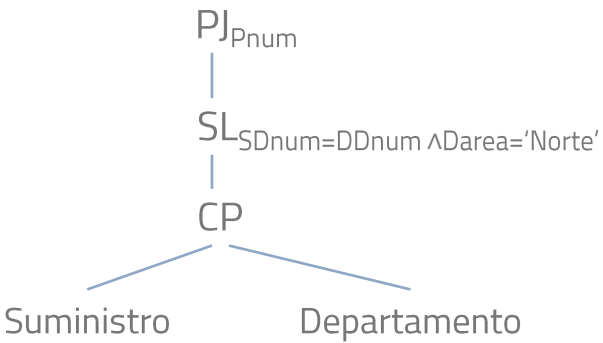
DESCOMPOSICIÓN DE CONSULTAS

Reescritura

Ejemplo:

Encontrar el número de proveedor de los proveedores que han suministrado artículos a departamentos del área Norte:

```
SELECT Pnum FROM Suministro S, Departamento D WHERE S.Dnum = D.Dnum AND D.area = 'Norte';
```



DESCOMPOSICIÓN DE CONSULTAS

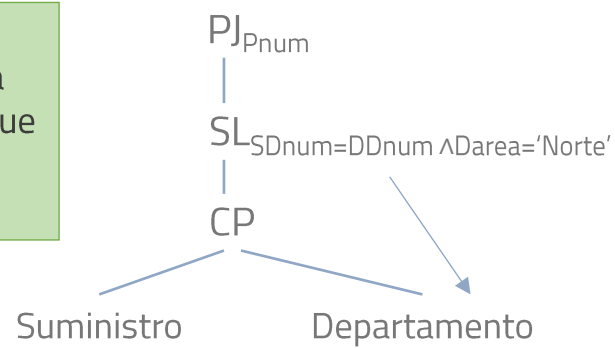
▶ Reescritura

▷ Ejemplo:

Encontrar el número de proveedor de los proveedores que han suministrado artículos a departamentos del área Norte:

```
SELECT Pnum FROM Suministro S, Departamento D WHERE S.Dnum = D.Dnum AND D.area = 'Norte';
```

Sería más apropiado hacer la operación de selección sobre la relación **Departamento** puesto que **área** es un atributo de dicha relación



Descomposición de consultas

Reescritura

- Dos relaciones son equivalentes (R ⇔ S) cuando representan la misma transformación de nombres de atributos en valores, independientemente del orden de esos atributos
- Ejemplo de relaciones equivalentes:

A	B
a1	b1
a2	b2
a3	b3

C	D
b1	a1
b2	a2
b3	a3

- Dos expresiones del álgebra relacional son equivalentes ($E_1 \Leftrightarrow E_2$) si los resultados de cada una son relaciones equivalentes

DESCOMPOSICIÓN DE CONSULTAS

- **Reescritura -** Reglas de transformación

Sean R, S y T relaciones

R definida sobre los atributos $A=\{A_1,A_2,\ldots,A_n\}$

S definida sobre $B=\{B_1,B_2,\ldots,B_n\}$

T definida sobre $C=\{C_1,C_2,\ldots,C_n\}$

1. Conmutatividad de operadores binarios :

$$R \ominus S \Leftrightarrow S \ominus R, \ominus = JN, CP, UN, INT$$

2. Asociatividad de operadores binarios:

$$(R \ominus S) \ominus T \Leftrightarrow R \ominus (S \ominus T), \ominus = JN, CP, UN, INT$$

DESCOMPOSICIÓN DE CONSULTAS

Reescritura - Reglas de transformación

3. Idempotencia de operadores unarios, aplicar el operador varias veces no cambia el resultado:

Proyección Si $A' \subseteq A$ y $A' \subseteq A''$, $\longrightarrow \pi_{A'}(\pi_{A''}(R)) \Leftrightarrow \pi_{A'}(R)$

Aplicar dos veces la proyección es innecesario si la segunda proyección es sobre un subconjunto de la primera.

```
SELECT nombre FROM (SELECT nombre, edad FROM Empleado);
```
⟶
```
SELECT nombre FROM Empleado;
```

Selección $\sigma_{p_i}(\sigma_{p_j}(R)) \Leftrightarrow \sigma_{p_i \wedge p_j}(R)$

Aplicar dos filtros consecutivos es equivalente a aplicar un solo filtro en ambas condiciones combinadas (AND)

```
SELECT * FROM (SELECT * FROM Empleado WHERE edad > 30) WHERE salario > 40000;
```

```
SELECT * FROM Empleado WHERE edad > 30 AND salario > 40000;
```

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

4. Conmutatividad de la selección con la proyección :

$$\pi_{A_1,\ldots,A_n}(\sigma_{p(A_p)}(R)) \Leftrightarrow \pi_{A_1,\ldots,A_n}(\sigma_{p(A_p)}(\pi_{A_1,\ldots,A_n,A_p}(R)))$$

Normalmente, **una selección y una proyección pueden conmutar** sin alterar el resultado final **SOLO SI** la selección no depende de atributos eliminados de la proyección.

```sql
SELECT Nombre
FROM (SELECT Nombre, Departamento FROM Empleado) AS Subconsulta
WHERE Salario > 3000;
```

¡Error! Salario no está en la proyección y no puede usarse en el WHERE

```sql
SELECT Nombre
FROM (SELECT Nombre, Departamento, Salario FROM Empleado) AS Subconsulta
WHERE Salario > 3000;
```

Ahora Salario está disponible en el WHERE

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

5. Conmutación de la selección con producto cartesiano: $\sigma_{p(A_i)}(R \times S) \Leftrightarrow (\sigma_{p(A_i)}(R)) \times S$

Si el predicado de selección afecta solo a la relación R, podemos aplicarlo antes de hacer el producto cartesiano, lo que reduce el número de combinaciones innecesarias, mejorando la eficiencia

```
SELECT *
FROM Empleado, Departamento
WHERE Empleado.DNI = '12345678A' AND Empleado.Dnum = Departamento.Dnum;
```

Primero se hace el producto cartesiano, lo que genera combinaciones necesarias

```
SELECT *
FROM (SELECT * FROM Empleado WHERE DNI = '12345678A') AS E, Departamento
WHERE E.Dnum = Departamento.Dnum;
```

Se filtra empleado antes de hacer el producto cartesiano

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

6. Conmutación de la selección con la unión: $\sigma_{p(A_i)}(R \bowtie_{p(A_i,B_k)} S) \Leftrightarrow (\sigma_{p(A_i)}(R)) \bowtie_{p(A_i,B_k)} S$

Si la condición de selección solo afecta a R, podemos aplicarla antes del JOIN, reduciendo filas antes de la combinación, lo que optimiza la consulta y mejor el rendimiento

```sql
SELECT *
FROM Empleado E
JOIN Departamento D ON E.Dnum = D.Dnum
WHERE E.Salario > 3000;
```

Primero hace el JOIN completo, luego filtra por Salario

```sql
SELECT *
FROM (SELECT * FROM Empleado WHERE Salario > 3000) AS E
JOIN Departamento D ON E.Dnum = D.Dnum;
```

Se filtran los empleados antes del JOIN, reduciendo el número de combinaciones

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

$$\sigma_{p(A_i)}(R \cup S) \Leftrightarrow \sigma_{p(A_i)}(R) \cup \sigma_{p(A_i)}(S)$$

$$\sigma_{p(A_i)}(R - S) \Leftrightarrow \sigma_{p(A_i)}(R) - \sigma_{p(A_i)}(S)$$

7. Conmutación de la selección con la unión o diferencia:

Podemos aplicar la selección antes o después de la unión o diferencia sin alterar el resultado. En bases de datos distribuidas, es mejor aplicar selección antes para reducir la cantidad de datos a transferir.

```sql
SELECT *
FROM (
    SELECT * FROM Empleado
    UNION
    SELECT * FROM ExEmpleado
) AS Temp
WHERE Salario > 3000;
```

Se unen todas las filas antes de filtrar

```sql
SELECT * FROM Empleado WHERE Salario > 3000
UNION
SELECT * FROM ExEmpleado WHERE Salario > 3000;
```

Se filtra antes de la unión, reduciendo el número de filas procesadas

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

8. Conmutación de la proyección con el producto cartesiano:

$$\pi_C(R \times S) \Leftrightarrow \pi_{A'}(R) \times \pi_{B'}(S)$$

Si $C = A' \cup B'$, con $A' \subseteq A$ y $B' \subseteq B$.

Si tenemos todos los atributos necesarios después del producto cartesiano, es posible proyectar en cada relación antes del producto cartesiano. Eso reduce el tamaño de los datos procesados optimizando el cálculo.

```
SELECT Empleado.Nombre, Departamento.Nombre
FROM Empleado, Departamento;
```

Primero se hace el producto cartesiano, que es costoso

```
SELECT E.Nombre, D.Nombre
FROM (SELECT Nombre, Dnum FROM Empleado) AS E,
     (SELECT Nombre, Dnum FROM Departamento) AS D;
```

Se proyectan los atributos antes de hacer el producto cartesiano, reduciendo el coste computacional

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

9. Conmutación de la proyección con unión:

$$\pi_C(R \bowtie_{p(A_i,B_j)} S) \Leftrightarrow \pi_{A'}(R) \bowtie_{p(A_i,B_j)} \pi_{B'}(S)$$

Si $C = A' \cup B'$, con $A' \subseteq A$, $B' \subseteq B$ y los atributos de unión no se pierden.

Podemos proyectar antes o después de la unión (JOIN) sin perder información, siempre que los atributos de la condición de unión se conserven. Si eliminamos atributos claves antes del JOIN, se puede perder información.

```
SELECT E.Nombre, D.Nombre
FROM (SELECT Nombre FROM Empleado) AS E
JOIN (SELECT Nombre FROM Departamento) AS D
ON E.Dnum = D.Dnum;
```

Se ha eliminado Dnum antes del JOIN

```
SELECT Empleado.Nombre, Departamento.Nombre
FROM Empleado
JOIN Departamento ON Empleado.Dnum = Departamento.Dnum;
```

El JOIN se hace sin problema

DESCOMPOSICIÓN DE CONSULTAS

▶ **Reescritura -** Reglas de transformación

$$\pi_C(R \cup S) \Leftrightarrow \pi_C(R) \cup \pi_C(S)$$

$$\pi_C(R - S) \Leftrightarrow \pi_C(R) - \pi_C(S)$$

10. Conmutación de la proyección con unión o diferencia:

Si R y S tienen el mismo esquema, podemos proyectar antes o después de la unión o diferencia sin alterar el resultado. Es más eficiente proyectar antes para reducir la cantidad de datos procesados.

```sql
SELECT Nombre, Edad
FROM (
    SELECT * FROM Empleado
    UNION
    SELECT * FROM ExEmpleado
) AS Temp;
```

Se unen todas las filas con todos los atributos antes de proyectar, lo que es ineficiente

```sql
SELECT Nombre, Edad FROM Empleado
UNION
SELECT Nombre, Edad FROM ExEmpleado;
```

Se proyectan solo los atributos necesarios antes de la unión, reduciendo el tamaño de los datos

DESCOMPOSICIÓN DE CONSULTAS

▶ Heurísticas de transformación -> busca una solución buena (no óptima)

▷ Descomponer las operaciones de selección con predicados conjuntivos

- Si una selección tiene varias condiciones AND, podemos dividirla en varias selecciones más pequeñas. De esta forma aplicamos las selecciones más restrictivas primero.

```
SELECT * FROM Empleado WHERE Salario > 3000 AND Departamento = 'Ventas';
```

```
SELECT * FROM (SELECT * FROM Empleado WHERE Departamento = 'Ventas') AS E
WHERE E.Salario > 3000;
```

Se filtra primero Departamento = ‘ Ventas’, reduciendo el número de filas antes de aplicar Salario > 3000

▷ Desplazar cada selección tan abajo en el árbol como lo permitan los atributos del predicado

- Cada operación de selección debe de ejecutarse lo más cerca posible de su relación base para reducir el número de filas desde el principio.

$$\sigma_{Salario>3000}(Empleado \bowtie Departamento)$$

$$(\sigma_{Salario>3000}(Empleado)) \bowtie Departamento$$

Filtrar Empleado antes del JOIN redice el número de combinaciones necesarias

DESCOMPOSICIÓN DE CONSULTAS

- **Heurísticas de transformación-> busca una solución buena (no óptima)**
  - Readaptar los nodos hoja del árbol, de forma que las relaciones con selecciones más restrictivas se ejecuten primero
    - Si hay múltiples selecciones, primero deben de ejecutarse las más restrictivas. Esto reduce el tamaño de los datos antes de realizar las operaciones.

```
SELECT * FROM Empleado E JOIN Departamento D ON E.Dnum = D.Dnum
WHERE D.area = 'Norte' AND E.Salario > 3000;
```

Es mejor filtrar Departamento primero (D.área = 'Norte'), ya que es más restrictivo y después E.Salario > 3000

  - Combinar una operación de producto cartesiano con una selección subsiguiente, cuyo predicado represente una condición de reunión (JOIN)
    - Si hay un producto cartesiano seguido de una selección con una condición de unión, se debe reemplazar directamente por un JOIN

Todas las combinaciones posibles, aunque solo una fracción es útil

```
SELECT * FROM Empleado, Departamento
WHERE Empleado.Dnum = Departamento.Dnum;
```

Un JOIN no genera todas las combinaciones, busca las coincidencias y devuelve las filas

```
SELECT * FROM Empleado
JOIN Departamento ON Empleado.Dnum = Departamento.Dnum;
```

DESCOMPOSICIÓN DE CONSULTAS

▶ Heurísticas de transformación-> busca una solución buena (no óptima)

▷ Descomponer las listas de atributos de proyección y desplazarlas lo más bajo posible en el árbol

- Aplicar la proyección cuantos antes, eliminando columnas innecesarias desde el principio para reducir el tamaño de los datos procesados.

```
SELECT Nombre FROM (SELECT * FROM Empleado WHERE Salario > 3000);
```

```
SELECT Nombre FROM (SELECT Nombre FROM Empleado WHERE Salario > 3000);
```

Filtrar columnas antes reduce la carga de datos y mejora el rendimiento

▷ Identificar subárboles que representen grupos de operaciones que se puedan ejecutar una vez y reutilizar el resultado más veces

- Si una subconsulta se usa varias veces en la misma consulta, se puede materializar en una tabla temporal o vista para evitar recomputarla

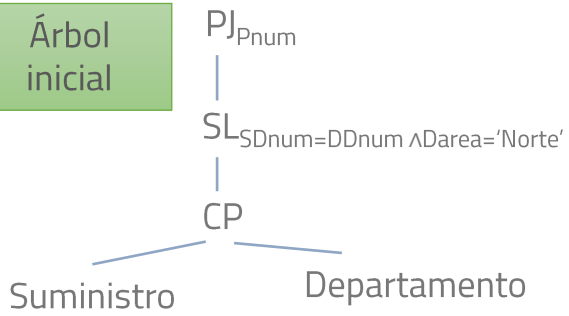
```
SELECT E.Nombre FROM Empleado E
WHERE E.Dnum IN (SELECT Dnum FROM Departamento WHERE area = 'Norte')
UNION
SELECT E.Nombre FROM ExEmpleado E
WHERE E.Dnum IN (SELECT Dnum FROM Departamento WHERE area = 'Norte');
```

```
WITH DepartamentosNorte AS (
    SELECT Dnum FROM Departamento WHERE area = 'Norte'
)
SELECT E.Nombre FROM Empleado E
WHERE E.Dnum IN (SELECT Dnum FROM DepartamentosNorte)
UNION
SELECT E.Nombre FROM ExEmpleado E
WHERE E.Dnum IN (SELECT Dnum FROM DepartamentosNorte);
```

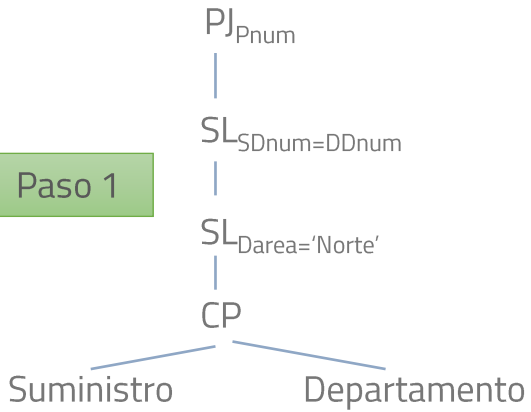
DESCOMPOSICIÓN DE CONSULTAS

- Ejemplo

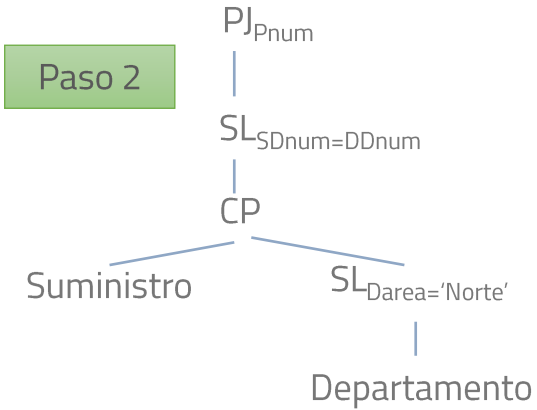
Árbol inicial



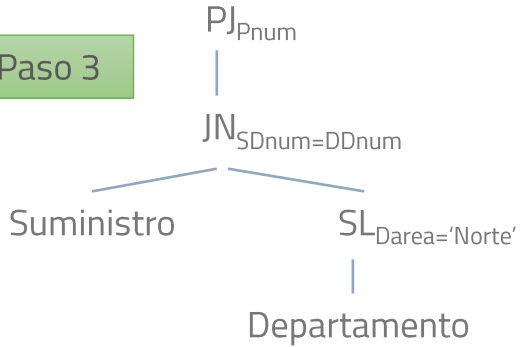
Paso 1



Paso 2



Paso 3



Árbol final

